

Laporan Hands-On 3: Process Lifecycle

NRP	Nama
5025251246	Hamizan Rifqi Afandi

Pada hands-on ini, konsep dasar process pada sistem operasi dibahas lewat serangkaian *shell script* yang bertujuan pada pembuatan process, observasi PID, monitoring status, komunikasi antarprocess, penganganan signal, dan dependency workflow.

Creating and Observing a Simple Process

Segmen ini memperkenalkan mengenai pembuatan process dan observasi PID untuk mengetahui identitasnya. Tujuan utama dari segmen ini yaitu menunjukkan bahwa sebuah process dapat dipindahkan ke background, diamaati statusnya, lalu diberhentikan melalui command pengguna.

File `task1_process_basic.sh` mengimplementasikan hal tersebut melalui dijalankannya perintah `sleep 30 &` untuk membawanya ke background selama 30 detik. Kemudian PID dari process tersebut di catat ke variabel `PID_BG` menggunakan `$!`, lalu diamati dengan command `ps -p -o .`. Script lalu menunggu sepanjang 3 detik, membunuh process tersebut menggunakan signal kill dengan command `kill`, lalu memeriksa kembali status process.

Kesimpulan

Dengan background execution, observasi PID, dan terminasi, mahasiswa dapat melihat bahwa process memiliki lifecycle yang dapat diikuti dan dikelola sejak tahap paling dasar.

Komponen	Isi
Filename	<code>task1_process_basic.sh</code>
Tools	<code>echo</code> , <code>sleep</code> , <code>\$!</code> , <code>ps</code> , <code>kill</code>

Snapshots Eksekusi



```

hamiz@hamiz-desktop:~/Documents/Sisop/hands-on$ mkdir hands-on-3
hamiz@hamiz-desktop:~/Documents/Sisop/hands-on$ ls
hands-on-3
hamiz@hamiz-desktop:~/Documents/Sisop/hands-on$ cd hands-on-3/
hamiz@hamiz-desktop:~/Documents/Sisop/hands-on/hands-on-3$ ls
hamiz@hamiz-desktop:~/Documents/Sisop/hands-on/hands-on-3$ touch task1_process_basic.sh
hamiz@hamiz-desktop:~/Documents/Sisop/hands-on/hands-on-3$ ls
task1_process_basic.sh
hamiz@hamiz-desktop:~/Documents/Sisop/hands-on/hands-on-3$ chmod +x task1_process_basic.sh
hamiz@hamiz-desktop:~/Documents/Sisop/hands-on/hands-on-3$ ls
task1_process_basic.sh
hamiz@hamiz-desktop:~/Documents/Sisop/hands-on/hands-on-3$ nano task1_process_basic.sh
hamiz@hamiz-desktop:~/Documents/Sisop/hands-on/hands-on-3$ ls
task1_process_basic.sh
hamiz@hamiz-desktop:~/Documents/Sisop/hands-on/hands-on-3$ ./task1_process_basic.sh
Running a sleep process in the background...
Background process PID: 104865
Displaying process status with ps:
  PID    PPID  STAT  CMD
  104865  104864  D+    /bin/bash ./task1_process_basic.sh
Waiting for 3 seconds...
Checking process status again:
  PID    PPID  STAT  CMD
  104865  104864  S+    sleep 30
Stopping the process...
Status after kill:
  PID    PPID  STAT  CMD
hamiz@hamiz-desktop:~/Documents/Sisop/hands-on/hands-on-3$

```

Monitoring the Status of Another Process

Dalam beberapa simple system, suatu process terkadang sering membutuhkan untuk mengetahui apabila suatu process masih berjalan, telah berhenti, atau telah gagal. Contoh nyatanya bisa dengan mengimplementasikan script monitoring untuk memastikan bahwa suatu service masih aktif.

File `task2_check_process_status.sh` mengimplementasikan hal tersebut melalui dijalankannya imitasi process worker `sleep 10 &` untuk membawanya ke background selama 10 detik. Kemudian PID dari process tersebut di catat ke variabel `WORKER_PID` menggunakan `#!`, lalu memeriksa keberadaan process dengan command `kill -0` dengan jeda 2 detik. Apabila worker masih berjalan, process mengoutput bahwa worker masih berjalan, sedangkan apabila tidak, process mengoutput bahwa telah berhenti dan keluar dari loop.

Kesimpulan

Status sebuah process dapat dipantau dari script lain tanpa harus menghentikan process tersebut. Dengan `kill -0`, process dapat melakukan cek keberadaan suatu process secara ringan dan menggunakan hasilnya untuk mengatur alur program. Segmen ini memperlihatkan dasar monitoring process yang nanti sangat berguna pada service supervision dan workflow automation.

Komponen	Isi
Filename	<code>task2_check_process_status.sh</code>

Komponen	Isi
Tools	<code>echo</code> , <code>sleep</code> , <code>#!</code> , <code>kill</code>

Snapshots Eksekusi

```

hamiz@hamiz-desktop: ~/Documents/Sisop/hands-on/hands-on-3$ ./task2
-bash: ./task2: No such file or directory
hamiz@hamiz-desktop: ~/Documents/Sisop/hands-on/hands-on-3$ ./task2_check_process_status.sh
Starting worker process...
Worker PID: 167984
[20:30:47] Worker is still running...
[20:30:49] Worker is still running...
[20:30:51] Worker is still running...
[20:30:53] Worker is still running...
[20:30:55] Worker is still running...
[20:30:57] Worker has finished.
Monitoring script finished.
hamiz@hamiz-desktop: ~/Documents/Sisop/hands-on/hands-on-3$

```

Interacting Between Processes Using a Signal File

Segmen ini memperkenalkan komunikasi antar proses menggunakan file sebagai media sederhana untuk koordinasi. Tujuan utamanya yaitu menunjukkan bahwa dua proses dapat berjalan bersamaan, saling berbagi informasi, dan melakukan sinkronisasi tanpa mekanisme komunikasi antar proses yang rumit seperti pipe atau socket.

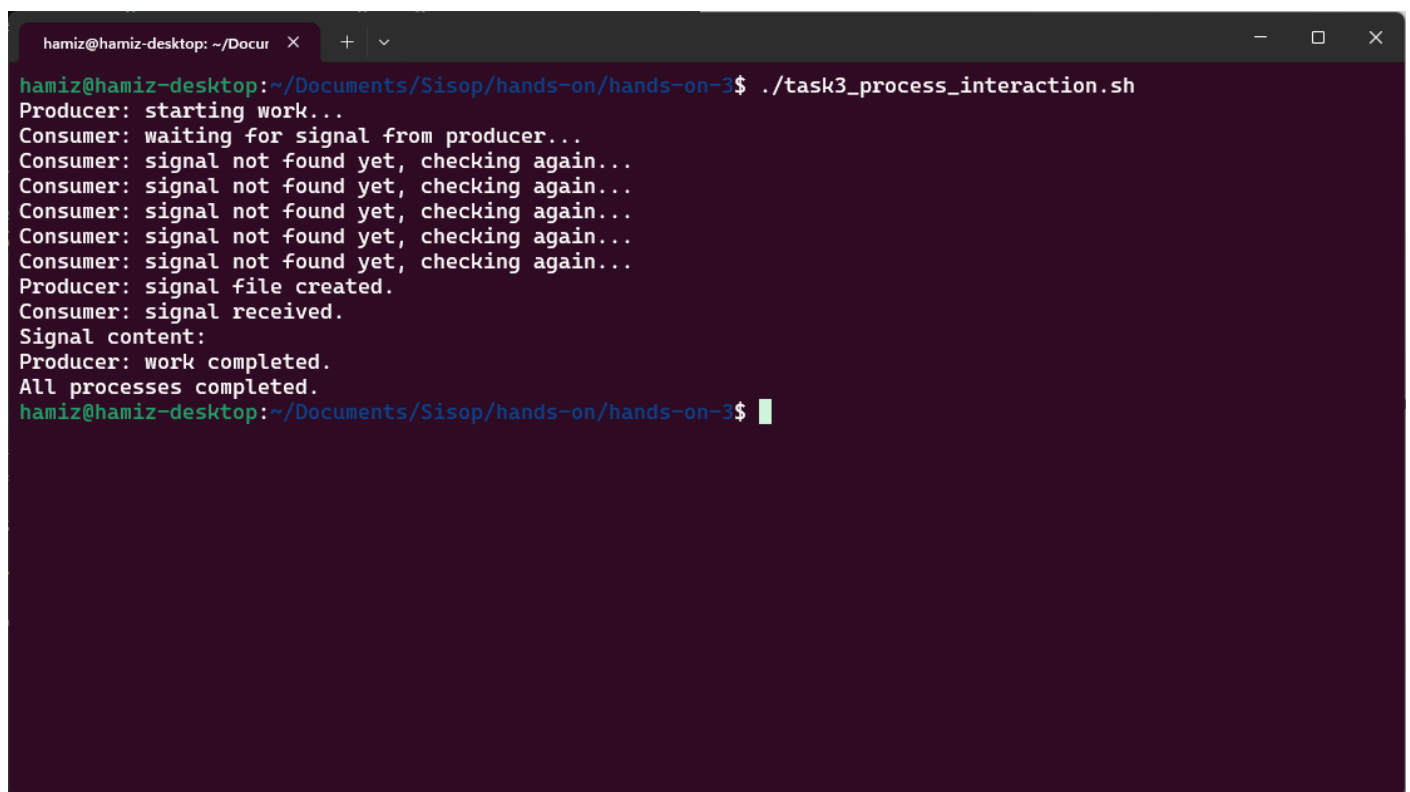
File `task3_process_interaction.sh` mengimplementasikan dua fungsi: `producer` dan `consumer`. `producer` mensimulasikan proses yang bekerja selama 5 detik, lalu membuat file sinyal di `/tmp/process_done.signal` sebagai tanda bahwa pekerjaannya selesai. `consumer` secara periodik memeriksa keberadaan file tersebut menggunakan loop `while [ ! -f "$SIGNAL_FILE" ]`. Ketika file ditemukan, `consumer` membaca isinya dan menampilkannya. Kedua proses dijalankan sebagai background process (`producer &` dan `consumer &`), kemudian `wait` digunakan untuk memastikan keduanya selesai sebelum skrip membersihkan file sinyal.

Kesimpulan

Dengan pendekatan file-based signaling, mahasiswa dapat memahami bahwa komunikasi antar proses tidak selalu membutuhkan mekanisme yang rumit. File system dapat berfungsi sebagai media komunikasi sederhana yang efektif untuk koordinasi dan sinkronisasi dasar antara proses yang berjalan secara konkuren.

Komponen	Isi
Filename	task3_process_interaction.sh
Tools	echo , sleep , rm , & , wait , cat , file system (/tmp)

Snapshots Eksekusi



```
hamiz@hamiz-desktop: ~/Docur x + v
hamiz@hamiz-desktop:~/Documents/Sisop/hands-on/hands-on-3$ ./task3_process_interaction.sh
Producer: starting work...
Consumer: waiting for signal from producer...
Consumer: signal not found yet, checking again...
Consumer: signal not found yet, checking again...
Consumer: signal not found yet, checking again...
Consumer: signal not found yet, checking again...
Consumer: signal not found yet, checking again...
Producer: signal file created.
Consumer: signal received.
Signal content:
Producer: work completed.
All processes completed.
hamiz@hamiz-desktop:~/Documents/Sisop/hands-on/hands-on-3$
```

Observing the Process Lifecycle (start, running, stop, terminated)

Segmen ini memperkenalkan siklus hidup proses secara lengkap, mulai dari proses dibuat, berjalan, menerima sinyal, hingga dihentikan. Tujuan utamanya yaitu menunjukkan bahwa sebuah proses tidak hanya "berjalan" atau "berhenti", tetapi memiliki tahapan hidup yang dapat diamati dan dikendalikan, termasuk kemampuan untuk melakukan pembersihan (cleanup) sebelum terminasi.

File `task4_process_lifecycle.sh` mengimplementasikan proses dengan infinite loop yang

mensimulasikan proses aktif berjalan terus menerus. Script ini menggunakan `trap cleanup SIGTERM SIGINT` untuk menangkap sinyal penghentian (SIGTERM dari `kill` atau SIGINT dari `Ctrl+C`). Ketika sinyal diterima, fungsi `cleanup()` dipanggil untuk melakukan pembersihan sebelum proses keluar. Variabel `$$` digunakan untuk menampilkan PID dari proses script itu sendiri. Dari terminal terpisah, pengguna dapat mengamati status proses menggunakan `ps -p <PID> -o pid,ppid,stat,cmd` dan mengirim sinyal terminasi dengan `kill -TERM <PID>`.

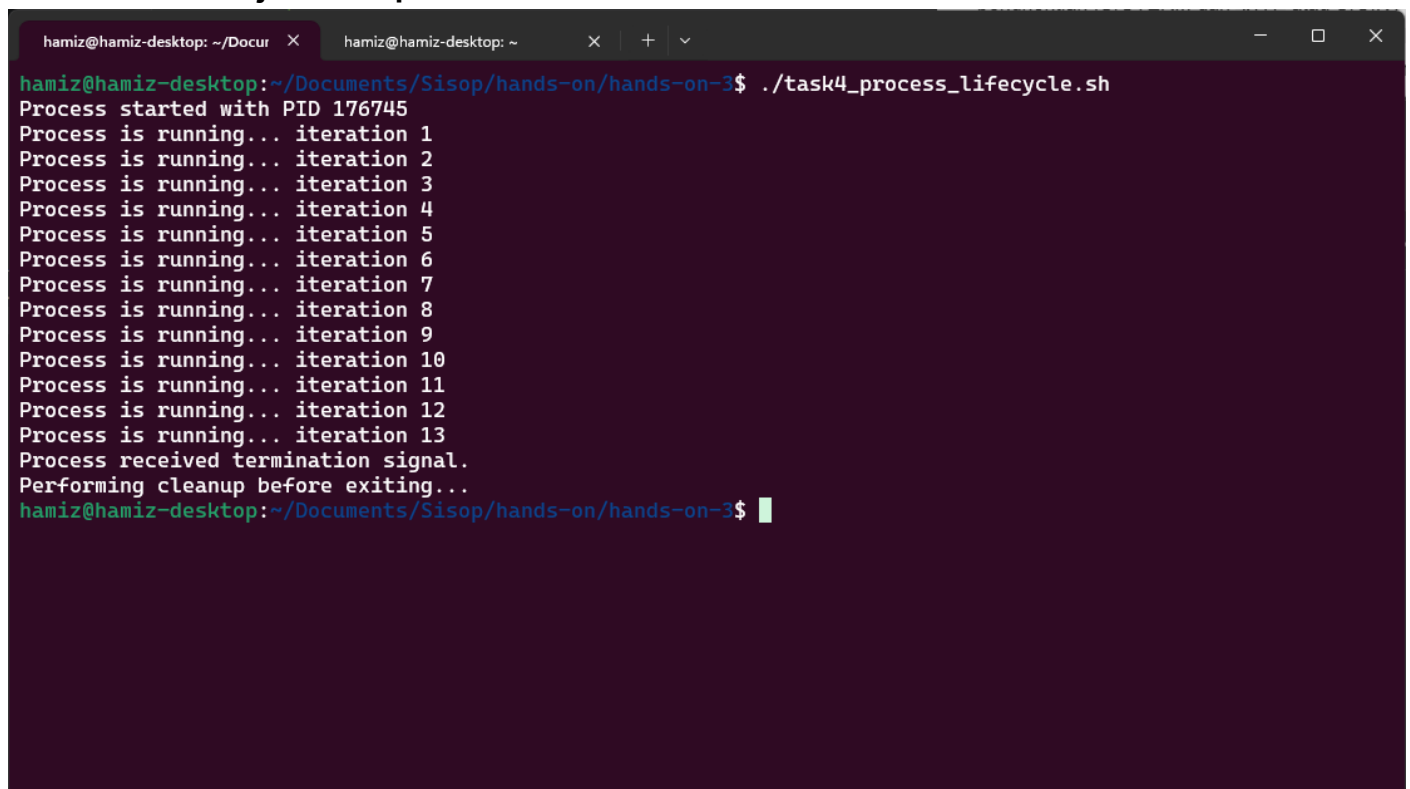
Kesimpulan

Dengan mekanisme signal handling menggunakan `trap`, mahasiswa dapat memahami bahwa proses memiliki siklus hidup yang dapat diobservasi dan dikendalikan secara eksplisit. Konsep cleanup sebelum terminasi sangat penting karena menunjukkan bahwa menghentikan proses dengan benar seringkali membutuhkan tindakan pembersihan, bukan sekadar mematikannya secara paksa.

Komponen	Isi
Filename	<code>task4_process_lifecycle.sh</code>
Tools	<code>trap</code> , <code>\$\$</code> , <code>echo</code> , <code>sleep</code> , <code>ps</code> , <code>kill</code>

Snapshots Eksekusi

Terminal 1 - Menjalankan proses:



```
hamiz@hamiz-desktop: ~/Documents/Sisop/hands-on/hands-on-3$ ./task4_process_lifecycle.sh
Process started with PID 176745
Process is running... iteration 1
Process is running... iteration 2
Process is running... iteration 3
Process is running... iteration 4
Process is running... iteration 5
Process is running... iteration 6
Process is running... iteration 7
Process is running... iteration 8
Process is running... iteration 9
Process is running... iteration 10
Process is running... iteration 11
Process is running... iteration 12
Process is running... iteration 13
Process received termination signal.
Performing cleanup before exiting...
hamiz@hamiz-desktop: ~/Documents/Sisop/hands-on/hands-on-3$
```

Terminal 2 - Observasi dan pengiriman sinyal:



```
hamiz@hamiz-desktop: ~$ ps -p 176745 -o pid,ppid,stat,cmd
PID    PPID  STAT  CMD
176745  103793 S+    /bin/bash ./task4_process_lifecycle.sh
hamiz@hamiz-desktop: ~$ kill -TERM 176745
hamiz@hamiz-desktop: ~$
```

Inter-Process Dependency in a Simple Workflow

Segmen ini memperkenalkan konsep dependensi antar proses dalam sebuah alur kerja (workflow). Tujuan utamanya yaitu menunjukkan bahwa suatu proses mungkin bergantung pada hasil dari proses lain, dan Bash dapat digunakan untuk mengatur urutan eksekusi serta memeriksa keberhasilan setiap langkah sebelum melanjutkan ke langkah berikutnya.

File `task5_process_dependency.sh` mengimplementasikan tiga langkah dalam sebuah workflow: `download_data` (mengumpulkan data), `validate_data` (memvalidasi data), dan `generate_report` (membuat laporan). Proses `download_data` dijalankan di background dengan PID-nya dicatat, kemudian `wait $PID_DOWNLOAD` digunakan untuk memastikan proses pengumpulan data selesai sepenuhnya sebelum melanjutkan. Setelah itu, `validate_data` memeriksa apakah file data tersedia dan tidak kosong, mengembalikan exit status (`0` untuk sukses, `1` untuk error). Jika validasi berhasil (`$? -eq 0`), maka `generate_report` dijalankan untuk membuat laporan; jika gagal, workflow berhenti dan menampilkan pesan error.

Kesimpulan

Dengan mekanisme `wait` untuk sinkronisasi dependensi dan pengecekan exit status untuk validasi hasil, mahasiswa dapat memahami bahwa Bash mampu membangun workflow yang terstruktur, aman, dan memiliki kontrol ketergantungan yang jelas antar proses. Konsep ini menjadi fondasi penting untuk memahami otomatisasi sistem, job scheduling, dan pipeline data di lingkungan yang lebih kompleks.

Komponen	Isi
Filename	task5_process_dependency.sh
Tools	echo , sleep , wait , \$? , if-else , file system (/tmp), wc -l

Snapshots Eksekusi

```
hamiz@hamiz-desktop: ~/Docur × + v
hamiz@hamiz-desktop:~/Documents/Sisop/hands-on/hands-on-3$ ./task5_process_dependency.sh
Waiting for download process to finish...
Step 1: Collecting data...
Step 1 completed: data saved in /tmp/sample_data.txt
Step 2: Validating data...
Data is valid.
Step 3: Generating report...
Report created in /tmp/report.txt
Workflow completed successfully.
Report content:
3 /tmp/sample_data.txt
hamiz@hamiz-desktop:~/Documents/Sisop/hands-on/hands-on-3$
```